SESSION 1 · PARCOURS UTILISATEUR

Utiliser la plateforme Clever Cloud

Une application, une base de données, un cluster Kubernetes — vus du côté du développeur

Horacio Gonzalez · 23 octobre 2026 · 2 heures, surtout de la pratique

« MAIS NOUS N'ALLONS PAS UTILISER LA PLATEFORME… »

Chaque ticket que vous recevrez commence par un développeur qui fait ce que nous allons faire aujourd'hui.

Créer une application. Déployer. Lier une base de données. Récupérer un kubeconfig.

POURQUOI UNE FORMATION UTILISATEUR

Sans l’expérience de l’utilisateur, le support devine

Qualifier

Distinguer un problème de plateforme d’une erreur d’usage — le cœur du métier N1/N2.

Reproduire

Refaire ce que décrit l’utilisateur et poser les bonnes questions.

Expliquer et escalader

Donner la solution avec les mots de l’utilisateur ; rédiger une escalade N3 utile.

Opérer en connaissance de cause

Savoir lesquelles de vos actions les développeurs vont ressentir.

PROGRAMME

Deux heures, surtout de la pratique

Heure	Séquence	Format
0:00	Pourquoi cette session · vérification	Exposé · Lab 0
0:10	Modèle mental : PaaS ou Kubernetes managé	Exposé
0:25	Une application Python simple	Pratique · Lab 1
0:50	Add-ons : créer, lier, injecter	Pratique · Lab 2
1:10	Une app moderne : uv + PostgreSQL	Démo · Lab 3
1:20	Pause	—
1:25	Clever Kubernetes Engine	Démo + pratique · Lab 4
1:55	Synthèse et suite du programme	Exposé

LE MODÈLE MENTAL

Où passe la frontière entre la plateforme et vous ?

La première question du support, à chaque fois.

QUI FAIT QUOI

Application PaaS ou cluster CKE partagé

Couche	Application PaaS	Cluster CKE
Qui la crée	Chaque développeur, par appli	L'équipe plateforme : 1–2 partagés
Votre code	Vous (git push)	Vous (images, manifestes)
Config et secrets	Vous (variables d'env.)	Vous (ConfigMaps, Secrets)
Build et runtime	Plateforme	Vous (images de conteneurs)
Scalabilité	Plateforme, dans vos bornes	Vous (réplicas, node groups)
Load balancing, TLS	Plateforme	LB L4 : plateforme · TLS : vous
Control plane	—	Plateforme
VM, OS, matériel	Plateforme	Plateforme (remplace les VM en panne)
Données	Add-ons managés	Add-ons ou vos propres volumes

RESSOURCES

Tout vit dans une organisation

Organisation
membres · rôles · facturation

Applications
code + runtime + variables d'environnement + scalabilité

Add-ons
PostgreSQL, Redis, Cellar… liés aux applications

Clusters CKE
1–2 partagés par organisation, opérés par l'équipe plateforme

CYCLE DE VIE D'UN DÉPLOIEMENT

Chaque déploiement crée de nouvelles instances



Le contrôle échoue ? Le déploiement échoue et la version précédente continue de servir.

Cache de build : un restart réutilise le dernier build ; un rebuild repart de zéro.

QUATRE RÈGLES DU JEU

Ce que la plateforme attend d’une application

Le code dans Git

Un commit = un déploiement immuable. Rollback = redéployer un commit plus ancien.

La config en variables

Les vôtres, celles de la plateforme (CC_*), celles des add-ons. Modification → restart.

L’état dans les add-ons

Le disque de l’instance est éphémère : les données vont dans une base ou un stockage objet.

Écouter sur 0.0.0.0:8080

Les logs vont sur stdout ; la plateforme les collecte.

LAB 1 · 25 MIN

Python simple : trois fichiers, un réglage

app.py — une petite application Flask
requirements.txt — Flask, pg8000
.gitignore

Ni configuration serveur, ni Dockerfile, ni script de build. La plateforme détecte Python et sert l'application avec uWSGI derrière Nginx.

Seul réglage obligatoire :
CC_PYTHON_MODULE=app:app

```
git init && git add . && git commit -m "init"

clever create --type python \
--org {{ORG}} python-<nom>

clever env set CC_PYTHON_MODULE app:app
clever env set CC_PYTHON_VERSION 3.13

clever deploy
clever domain
```

LAB 1 · LIRE LE JOURNAL DE DÉPLOIEMENT

Le journal raconte l’histoire — lisez-le d’abord

Trouvez la ligne de chaque étape :

- le commit déployé
- la version de Python choisie
- l’installation des dépendances
- l’envoi du cache de build
- le contrôle du port 8080
- le succès du déploiement

```
clever activity
clever instances
clever logs --since 10m
clever accesslogs --since 10m

curl https://<domaine-app>/
```

LAB 1 · NOUVELLE VERSION

Blue/green : la nouvelle version démarre avant que l'ancienne s'arrête




```
while true; do curl -s https://<domaine-app>/ | grep -o '"version":"[^"]*"'; sleep 1; done
```

LAB 2 · 20 MIN

Add-ons : des services managés à côté des applications

Bases de données, caches, stockage objet, systèmes de fichiers, identité, analytics… Chaque fournisseur a des **plans** (taille) et des **zones**.

La plateforme les provisionne, les supervise et les sauvegarde. Vous choisissez le plan et possédez les données.

Que propose **votre** installation ? **clever addon providers**

```
clever addon providers
clever addon providers \
show postgresql-addon

clever addon create \
postgresql-addon pg-<nom> \
--plan dev --org {{ORG}}
clever service link-addon pg-<nom>
```

LAB 2 · LIER

Lier injecte les variables de l’add-on au démarrage de l’application



Une instance déjà démarrée ne les voit pas → restart. Le ticket n°1 : « mon application ne voit pas la base ».

LAB 3 · DÉMO · 10 MIN

Une app moderne : Django, uv et PostgreSQL

github.com/CleverCloud/django-todo-list-example — observez, puis répondez :

1. Quel fichier fait passer la plateforme en mode uv ? Quelle variable faut-il aussi ?
2. Pourquoi CC_PYTHON_MODULE n'est-il pas nécessaire ici ?
3. D'où viennent les identifiants de la base de données ?
4. Les migrations tournent dans CC_PRE_RUN_HOOK. Que peut-il se passer avec 3 instances ?
5. « Mon application répond Bad Request (400) » — que vérifiez-vous ?

LAB 3 · UN RUNTIME, DEUX MODES

C’est le projet qui décide comment Python tourne

	Mode classique	Mode uv natif
Déclenché par	requirements.txt	uv.lock + CC_PYTHON_UV_RUN_COMMAND
Dépendances	pip install	uv sync --locked
Serveur HTTP	Nginx + uWSGI (ou gunicorn…)	Le vôtre (uvicorn…)
Point d’entrée	CC_PYTHON_MODULE	CC_PYTHON_UV_RUN_COMMAND
Port	géré par la plateforme	votre serveur sur 0.0.0.0:8080

Réflexe support : savoir d’abord quel mode s’applique — le même symptôme a des causes différentes.

LAB 4 · 30 MIN

Clever Kubernetes Engine

Clever Cloud opère le control plane et remplace les VM en panne.
L’équipe plateforme opère un ou deux clusters partagés.
Les développeurs déploient dans leur namespace — avec kubectl.

En bêta · activé par utilisateur · en ligne de commande pour l’instant (console à venir)

Trois topologies, choisies à la création

Topologie	Organisation	Usage type
ALL_IN_ONE (défaut)	Control plane + worker sur les mêmes VM	Dev, test, petites équipes
DEDICATED_COMPUTE	VM de control plane + node groups séparés	Production
DISTRIBUTED	Une VM par composant du control plane + node groups	Production exigeante

Aussi : flavor, facteur de réplication (1–5), version de Kubernetes, node groups, autoscaling (optionnel), stockage persistant (Ceph CSI). **Topologie et facteur de réplication ne changent plus ensuite.**

```
clever k8s create --help
clever k8s create {{CLUSTER}} \
--org {{ORG}} --watch
```

LAB 4 · PRATIQUE

Un cluster partagé, un namespace chacun — comme au travail

1 · SE CONNECTER

```
clever k8s get-kubeconfig \
{{CLUSTER}} --org {{ORG}} \
> ~/.kube/cc-training.yaml
export KUBECONFIG=\
~/.kube/cc-training.yaml
kubectl get nodes
```

2 · DÉPLOYER

```
kubectl create ns <nom>
kubectl apply -n <nom> \
-f whoami.yaml
kubectl get pods \
-n <nom> -o wide
```

3 · Y ACCÉDER

```
kubectl port-forward \
-n <nom> \
svc/whoami 8081:80

curl localhost:8081
```

⚠ Le kubeconfig donne un accès complet au cluster : traitez-le comme un secret. N'écrasez pas votre propre ~/.kube/config.

LAB 4 · SUPPRIMEZ UN POD, OBSERVEZ

Qui répare quoi — et qui reçoit le ticket

Kubernetes répare les pods

Le Deployment recrée un pod supprimé ou planté. Un pod en CrashLoopBackOff relève du workload de l'utilisateur.

```
kubectl logs · get events
```

Clever Cloud répare les VM

Une VM de control plane ou de worker en panne est remplacée automatiquement. Un nœud NotReady ou un API server injoignable relève de la plateforme.

```
clever k8s activity · get
```

À RETENIR

Six choses à retenir pour le prochain ticket

1. Trouver la frontière : plateforme ou utilisateur ? Application PaaS ou CKE ?
2. Chaque déploiement crée de nouvelles instances ; leur disque ne survit pas.
3. La configuration, ce sont des variables : les vôtres, CC_*, celles des add-ons. Modification → restart.
4. Les nouvelles instances passent les contrôles avant la bascule du trafic.
5. Lire le journal de déploiement en premier.
6. C’est le projet qui décide du mode (classique ou uv) : vérifier lequel s’applique.

LA SUITE

La prochaine fois, c’est vous qu’on appelle

Parcours	Session
Support N1	Support utilisateur et qualification des besoins
Support N1	Méthodologie de dépannage — casser, puis réparer
Support N1	Cas de support concrets
Support N2	Dépannage avancé et supervision
Support N2	Support utilisateur avancé
Support N2	Focus thématique : infrastructure

Avant de partir : nettoyage (Lab 5) — sauf si nous gardons les ressources pour les prochaines sessions.